Министерство науки и высшего образования
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Бинарное дерево поиска»

Выполнили: ст. гр. 25ВВВ2
Потёмкин Д.С.
Гладков А.М.

Проверил: к.п.н., доцент
Юрова О.В.
Деев М.В.

Пенза 2026 г.

Название: Бинарное дерево поиска

Цель работы: Изучение принципов построения бинарного дерева поиска на языке Си. Получение навыков работы с указателями и динамической памятью, реализации поиска и подсчёта повторяющихся значений, а также оценки сложности алгоритмов.

Лабораторное задание

1. Реализовать поиск вводимого с клавиатуры значения в уже созданном дереве.

2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.

3. Изменить функцию добавления элементов для исключения повторяющихся значений.

4. Оценить сложность процедуры поиска по значению в бинарном дереве.

Пояснительный текст к программе

Программа разработана на языке программирования Си с использованием стандартных библиотек. Библиотека stdio.h используется для ввода и вывода, stdlib.h — для выделения и освобождения памяти. Перед построением дерева выбирается режим: с повторами или без них.

Структура Node содержит следующие поля:

- data — целочисленное значение узла;
- left — указатель на левое поддерево;
- right — указатель на правое поддерево.

Указатель root хранит адрес корня; NULL обозначает пустое дерево. Все ключи слева меньше ключа текущего узла. Справа находятся большие или равные ключи; при запрете повторов — только большие.

Пояснения к Заданию 1

Переменные и параметры:

- root — указатель на текущий узел дерева;
- value — значение, введённое пользователем для поиска.

Основные операции функции Search:

- Сравнение value со значением текущего узла. При совпадении возвращается адрес найденного узла.
- Переход в левое поддерево, если value меньше текущего ключа; в правое — если больше.
- Возврат NULL, если достигнуто пустое поддерево. Это означает, что искомого значения в дереве нет.

Поиск выполняется циклом и не изменяет содержимое дерева. В main результат используется для вывода сообщения found или not found.

Пояснения к Заданию 2

Функция CountOccurrences получает указатель root и искомое значение value и возвращает количество совпадений.

Основные операции:

- Для пустого поддерева возвращается 0.
- При несовпадении поиск продолжается в одном поддереве, выбранном сравнением ключей.
- При совпадении возвращается 1 плюс число вхождений в правом поддереве.

Левое поддерево при совпадении не проверяется: все его ключи строго меньше текущего. Повтор может находиться глубже, например слева от большего ключа в правом поддереве; рекурсивные сравнения учитывают такой случай.

Пояснения к Заданию 3

Функция insert получает корень root, добавляемое число data и признак unique. CreateTree передаёт unique = 0, а CreateTreeUnique — unique = 1.

Основные операции:

- При NULL выделяется память для нового узла, записывается data, а ссылки на потомков обнуляются.
- Меньшее значение добавляется слева, большее — справа. Возвращаемый указатель сохраняется у родителя.
- Равное значение добавляется справа только при unique = 0. В режиме unique = 1 новый узел не создаётся.

Пояснения к Заданию 4

Пусть n — количество узлов, h — высота непустого дерева. Поиск проверяет узлы одного пути и имеет сложность O(h).

- В сбалансированном дереве h = O(log n), время поиска — O(log n).
- В худшем случае дерево образует цепочку: h = n, время поиска — O(n).
- При совпадении с корнем поиск занимает O(1).

Средняя оценка O(log n) справедлива, например, при случайном порядке вставки различных ключей. Произвольный ввод её не гарантирует: автоматическая балансировка в программе отсутствует.

Search использует O(1) дополнительной памяти. Подсчёт вхождений также идёт по одному пути за O(h), но требует O(h) памяти для рекурсивных вызовов. Для хранения всего дерева используется O(n) памяти.

Завершение и освобождение памяти

Число -1 завершает построение и не добавляется в дерево. После поиска функция FreeTree рекурсивно освобождает потомков и текущий узел. Результат каждого вызова scanf проверяется; при неверном вводе программа сообщает об ошибке.

Результаты работы программы

Для первых трёх проверок введены числа: 8, 3, 10, 1, 6, 14, 4, 7, 13, 6, 6. После них введено -1. На рисунках приведён вывод выполненной программы.

В режиме 1 число 6 найдено; количество вхождений равно 3. Результат показан на рисунке 1.

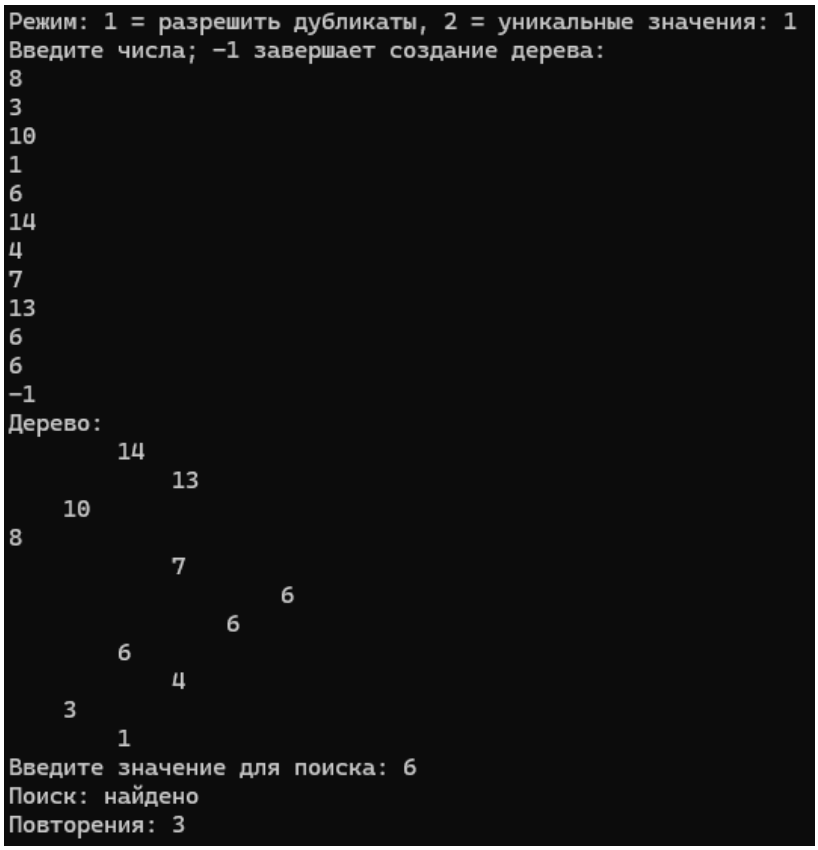


Рисунок 1

При поиске числа 5 в том же дереве получены not found и 0 вхождений. Результат показан на рисунке 2.

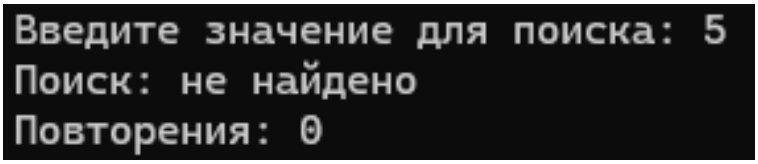


Рисунок 2

В режиме 2 повторы исключены: число 6 встречается один раз. Результат показан на рисунке 3.

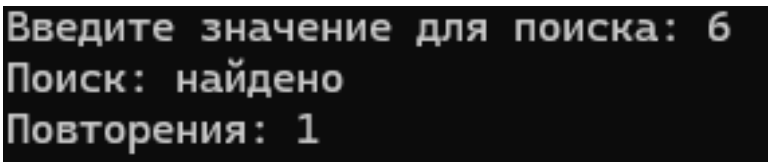


Рисунок 3

При завершении ввода без добавления узлов дерево пустое. Поиск числа 8 возвращает not found и 0 вхождений. Результат показан на рисунке 4.

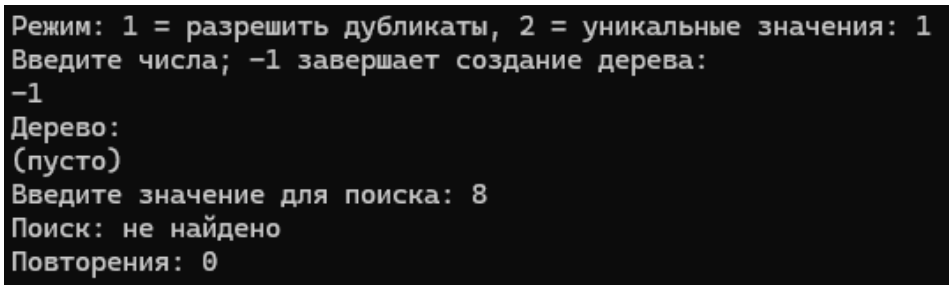


Рисунок 4

Выводы

В ходе выполнения лабораторной работы разработана программа на языке Си, реализующая построение бинарного дерева поиска и обработку его узлов.

Задание 1: Реализован итеративный поиск заданного числа. Функция возвращает указатель на найденный узел или NULL при отсутствии значения.

Задание 2: Реализован рекурсивный подсчёт вхождений. Для набора с тремя значениями 6 получен результат 3; при отсутствии искомого числа — 0.

Задание 3: Реализован режим добавления без дубликатов. Повторный ввод существующего значения не приводит к созданию нового узла.

Задание 4: Установлено, что сложность поиска равна O(h). Для сбалансированного дерева она составляет O(log n), для вырожденного — O(n).

Программа скомпилирована GCC в режиме C11 без предупреждений. Выполнены 8 основных, 200 дополнительных проверок со случайными наборами и 4 проверки неверного ввода. Проверенные результаты совпали с ожидаемыми.

Получены навыки работы со структурами, указателями, рекурсией и динамической памятью, а также реализации и анализа алгоритмов обработки бинарного дерева поиска.